

Benemérita Universidad Autónoma de Puebla

Facultad de Ciencias de la Computación

Evidencia de aprendizaje: Análisis semanal de mediciones de glucosa

Santiago Adrian Lazos Ovalle

ID de Estudiante: 202522695 https://github.com/SantiagoLazos/Estructura_de_Datos

Materia:	Estructuras de Datos y sus Aplicaciones
Fecha de entrega:	8 de septiembre de 2026
Periodo:	Otoño 2026

Descripción breve del funcionamiento del algoritmo

El programa inicia definiendo una estructura base de **Cola** que se encarga de manejar la capacidad y el arreglo principal. A partir de esta, mediante herencia, se construye la clase **ListaEstatica**, la cual simula el comportamiento de una estructura estática en memoria limitando el acceso a un tamaño fijo de 7 espacios (los días de la semana). Se utiliza también una clase **Persona** para encapsular tanto la identidad del paciente como su propia instancia de lista de mediciones, logrando así un alto nivel de modularidad.

El algoritmo recibe las mediciones de cada persona y evalúa los datos utilizando funciones específicas para calcular el valor máximo, el valor mínimo y determinar el estatus de salud. El estatus se evalúa recorriendo los elementos válidos de la lista estática; si alguna medición sale del rango ideal de 70 a 140 mg/dL, el sistema rompe el ciclo y clasifica a la persona inmediatamente como DESCONTROLADO.

1. Código Fuente del Programa

1.1 Clase de lista estática utilizada en la solución (y Herencia de Cola)

```
1  from Cola import *
2
3  class ListaEstatica(Cola):
4      """
5      Hereda de la clase Cola para reutilizar self.vector, self.capacidad y self.fin.
6      Le agregamos las operaciones específicas de una Lista que pide el documento.
7      """
8      def __init__(self, capacidad):
9          super().__init__(capacidad) # Llama al __init__ de tu Cola original
10
11      # 1. Agregar una medición (Usamos la lógica de tu encolar)
12      def agregar(self, elemento):
13          # En tu clase Cola original esto es encolar()
14          self.encolar(elemento)
15
16      # 2. Consultar una medición mediante su posición
17      def consultar(self, posicion):
18          if 0 <= posicion <= self.fin:
19              return self.vector[posicion]
20          else:
21              print("Error: Posición inválida.")
22              return None
23
24      # 3. Buscar una medición determinada
25      def buscar(self, elemento):
26          for i in range(self.fin + 1):
27              if self.vector[i] == elemento:
28                  return i # Retorna la posición
29          return -1 # No encontrado
30
31      # 4. Modificar una medición existente
32      def modificar(self, posicion, nuevo_elemento):
33          if 0 <= posicion <= self.fin:
34              self.vector[posicion] = nuevo_elemento
35              return True
36          print("Error: Posición inválida.")
37          return False
38
39      # 5. Eliminar una medición (Rompe la regla FIFO de la cola, pero es necesario para la Lista)
40      def eliminar(self, posicion):
41          if self.esVacio():
42              print("Error: La lista está vacía.")
43              return False
44          if 0 <= posicion <= self.fin:
45              # Desplazamos a la izquierda para tapar el hueco
46              for i in range(posicion, self.fin):
47                  self.vector[i] = self.vector[i + 1]
48              self.vector[self.fin] = None
49              self.fin -= 1
50              return True
51          print("Error: Posición inválida.")
52          return False
53
54      def obtener_elementos_validos(self):
55          # Devuelve solo el fragmento del vector que tiene datos
56          if self.esVacio():
57              return []
58          return self.vector[:self.fin + 1]
59
```

1.2 Clase Persona

```
class Persona:
    """Clase para identificar a la persona y almacenar su lista de mediciones."""
    def __init__(self, nombre):
        self.nombre = nombre
        self.lista_mediciones = ListaEstatica(7) # Capacidad 7 días

    def registrar_medicion(self, valor):
        self.lista_mediciones.agregar(valor)

    def calcular_maxima(self):
        elementos = self.lista_mediciones.obtener_elementos_validos()
        if not elementos: return None
        maxima = elementos[0]
        for val in elementos:
            if val > maxima: maxima = val
        return maxima

    def calcular_minima(self):
        elementos = self.lista_mediciones.obtener_elementos_validos()
        if not elementos: return None
        minima = elementos[0]
        for val in elementos:
            if val < minima: minima = val
        return minima

    def determinar_estatus(self):
        elementos = self.lista_mediciones.obtener_elementos_validos()
        if not elementos: return "SIN DATOS"
        for val in elementos:
            if val < 70 or val > 140:
                return "DESCONTROLADO"
        return "CONTROLADO"

    def mostrar_analisis(self):
        print(f"\n--- Análisis de Glucosa: {self.nombre} ---")
        print(f"Mediciones (Lun-Dom): {self.lista_mediciones.obtener_elementos_validos()}")
        if not self.lista_mediciones.esVacio():
            print(f"Medición Mínima: {self.calcular_minima()}")
            print(f"Medición Máxima: {self.calcular_maxima()}")
            print(f"Estatus actual: {self.determinar_estatus()}")
```

1.3 Implementación (Main) y Casos de Prueba

```
def main():
    print("=== SISTEMA DE CONTROL DE GLUCOSA SEMANAL ===")

    # Carga de datos
    ana = Persona("Ana")
    for med in [92, 105, 110, 98, 115, 108, 100]: ana.registrar_medicion(med)

    carlos = Persona("Carlos")
    for med in [180, 175, 190, 165, 185, 172, 195]: carlos.registrar_medicion(med)

    luis = Persona("Luis")
    for med in [130, 145, 138, 150, 142, 135, 128]: luis.registrar_medicion(med)

    # Casos de prueba requeridos en el PDF
    print("\n[ CASO 1: Persona controlada (Ana) ]")
    ana.mostrar_analisis()

    print("\n[ CASO 2: Persona descontrolada (Carlos) ]")
    carlos.mostrar_analisis()

    print("\n[ CASO 3: Modificación (Luis: Jueves 150 -> 135) ]")
    luis.mostrar_analisis()
    luis.lista_mediciones.modificar(3, 135)
    print("-> Después de modificar:")
    luis.mostrar_analisis()

    print("\n[ CASO 4: Eliminación (Carlos: Domingo) ]")
    carlos.mostrar_analisis()
    carlos.lista_mediciones.eliminar(6)
    print("-> Después de eliminar (Domingo):")
    carlos.mostrar_analisis()

    print("\n[ CASO 5: Búsqueda ]")
    print(f"Ana - Posición del valor 110: {ana.lista_mediciones.buscar(110)}")
    print(f"Ana - Posición del valor 200: {ana.lista_mediciones.buscar(200)}")

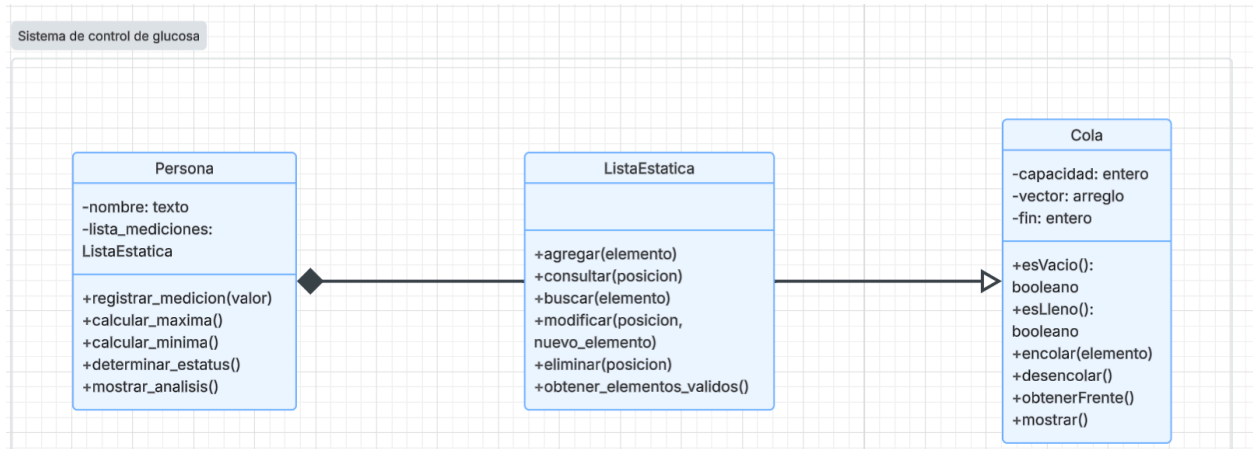
    print("\n[ CASO 6: Operaciones con Lista Vacía ]")
    paciente_vacio = Persona("Prueba Vacía")
    paciente_vacio.lista_mediciones.consultar(0)
    paciente_vacio.lista_mediciones.eliminar(0)
    paciente_vacio.mostrar_analisis()

if __name__ == "__main__":
    main()
```

2. Diagrama de Clases UML en Lucidchart

Lucidchart

Ya con el proyecto estructurado, procedí a generar el diagrama de clases correspondiente. A continuación, se presenta la representación visual de la relación de Herencia entre la Cola y la Lista Estática, así como la Composición con la clase Persona.



3. Evidencias de Ejecución del Programa

Al ejecutar el código fuente completo, el sistema procesa los 6 casos de prueba exigidos. Se verifican las validaciones, el manejo de la lista vacía, las modificaciones de estado y la alteración y eliminación de datos en tiempo de ejecución.

```
=== SISTEMA DE CONTROL DE GLUCOSA SEMANAL ===
```

```
[ CASO 1: Persona controlada (Ana) ]
```

```
--- Análisis de Glucosa: Ana ---
```

```
Mediciones (Lun-Dom): [92, 105, 110, 98, 115, 108, 100]
```

```
Medición Mínima: 92
```

```
Medición Máxima: 115
```

```
Estatus actual: CONTROLADO
```

```
[ CASO 2: Persona descontrolada (Carlos) ]
```

```
--- Análisis de Glucosa: Carlos ---
```

```
Mediciones (Lun-Dom): [180, 175, 190, 165, 185, 172, 195]
```

```
Medición Mínima: 165
```

```
Medición Máxima: 195
```

```
Estatus actual: DESCONTROLADO
```

```
[ CASO 3: Modificación (Luis: Jueves 150 -> 135) ]
```

```
--- Análisis de Glucosa: Luis ---
```

```
Mediciones (Lun-Dom): [130, 145, 138, 150, 142, 135, 128]
```

```
Medición Mínima: 128
```

```
Medición Máxima: 150
```

```
Estatus actual: DESCONTROLADO
```

```
-> Después de modificar:
```

```
--- Análisis de Glucosa: Luis ---
```

```
Mediciones (Lun-Dom): [130, 145, 138, 135, 142, 135, 128]
```

```
Medición Mínima: 128
```

```
Medición Máxima: 145
```

```
Estatus actual: DESCONTROLADO
```

```
[ CASO 4: Eliminación (Carlos: Domingo) ]
```

```
--- Análisis de Glucosa: Carlos ---
```

```
Mediciones (Lun-Dom): [180, 175, 190, 165, 185, 172, 195]
```

```
Medición Mínima: 165
```

```
Medición Máxima: 195
```

```
Estatus actual: DESCONTROLADO
```

```
-> Después de eliminar (Domingo):
```

```
--- Análisis de Glucosa: Carlos ---
```

```
Mediciones (Lun-Dom): [180, 175, 190, 165, 185, 172]
```

```
Medición Mínima: 165
```

```
Medición Máxima: 190
```

```
Estatus actual: DESCONTROLADO
```

```
[ CASO 5: Búsqueda ]
```

```
Ana - Posición del valor 110: 2
```

```
Ana - Posición del valor 200: -1
```

```
[ CASO 6: Operaciones con Lista Vacía ]
```

```
Error: Posición inválida.
```

```
Error: La lista está vacía.
```

```
--- Análisis de Glucosa: Prueba Vacía ---
```

```
Mediciones (Lun-Dom): []
```

4. Conclusión

Al desarrollar esta aplicación, pude comprobar de primera mano lo útil que resulta implementar listas estáticas mediante arreglos de tamaño fijo. En un contexto clínico donde registramos datos sensibles como la glucosa, trabajar con una estructura rígida en memoria nos da mucha certeza sobre el comportamiento del sistema y nos ayuda a no desperdiciar recursos. Al fijar el tamaño del arreglo a siete posiciones (una por cada día de la semana), el uso de índices vuelve muy directas y eficientes todas las consultas, modificaciones y cálculos, como obtener las mediciones máximas y mínimas.

Por otro lado, separar la lógica de negocio usando programación orientada a objetos haciendo que cada objeto `Persona` gestione de forma independiente su propia lista de mediciones me permitió mantener el código muy modular y fácil de escalar. Además, aprovechar la herencia desde la clase `Cola` hacia `ListaEstatica` demostró cómo podemos reutilizar código base de forma limpia sin romper las reglas de estructura que requería el proyecto.

[https://github.com/SantiagoLazos/Estructura de Datos](https://github.com/SantiagoLazos/Estructura_de_Datos)